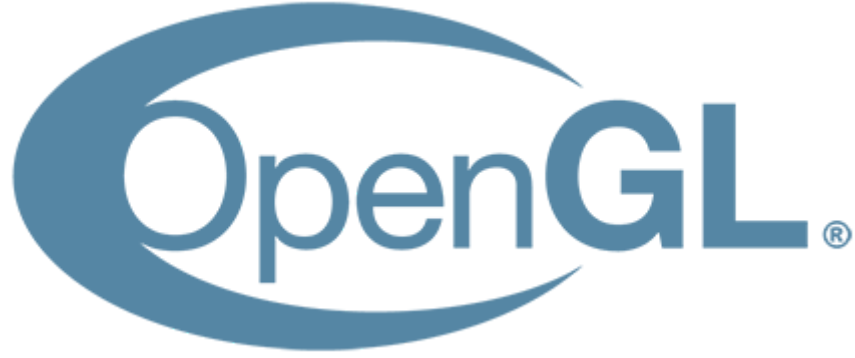




İstanbul Üniversitesi-Cerrahpaşa
Bilgisayar Grafikleri Projesi
Raporu

Ad-Soyad:

Mustafa Çirci

Üniversite:

İstanbul Üniversitesi-Cerrahpaşa

Numara:

1306210018

Tema

Tema olarak ilk bilgisayarımı ve küçüklükteki odamı tasarladım. Başta sadece oyun oynadığım bir cihazdı ama zamanla içinde neler olduğunu merak etmeye başladım. Ayarlarını kurcaladım, bozdum, yeniden kurdum... Bu merak, beni bilgisayarların sadece kullanıcı tarafında değil, nasıl çalıştıklarıyla da ilgilenmeye yöneltti. O günlerde farkında olmasam da, aslında bilgisayar mühendisliğine giden yolun ilk adımlarını atmamı sağladı.



Kullanılan Teknik Bileşenler ve Kütüphaneler

GLAD: OpenGL fonksiyonlarını dinamik olarak yüklemek için kullandım. Bu kütüphane ile gerekli olan tüm OpenGL işlevlerini yükledim.

GLFW: OpenGL penceresini oluşturmak ve kullanıcı etkileşimlerini tanımlamak için kullandım. Kameradaki etkileşimler için çokça kullandım.

GLM: Vektör ve matris hesaplamalarında kullandım. Özellikle matris dönüşümlerinde kolaylık sağladı.

STB_Image: Görselleri projeye entegre etmek için kullandım. STB_Image ile resimleri byte'a dönüştürüp texture olarak ilettim.

Classlar

VBO.cpp

OpenGL'de kullanılan Vertex Buffer Object (VBO) işlemlerini yöneten bir sınıftır. VBO, vertex verilerinin (örneğin pozisyon, normal, doku koordinatları) GPU'ya gönderilmesini ve rendering işlemlerinin gerçekleştirilmesini sağlar.

Constructor (VBO::VBO): Dışarıdan bir `std::vector<Vertex>` alınır ve bu veriler GPU'ya aktarılır.

- `glGenBuffers` ile bir VBO ID'si oluşturulur.
- `glBindBuffer` ile bu ID'ye sahip VBO `GL_ARRAY_BUFFER` hedefiyle bağlanır.
- `glBufferData` fonksiyonu ile vertex verileri GPU'ya kopyalanır.

Bind(): VBO'yu `GL_ARRAY_BUFFER` olarak bağlar. Bu işlem, VBO'nun aktif hale gelmesini sağlar.

Unbind(): `glBindBuffer` fonksiyonu ile 0 ID'si atanarak aktif VBO çözülür. Bu, VBO'nun geçici olarak devre dışı bırakılması anlamına gelir.

Delete(): `glDeleteBuffers` ile oluşturulan VBO silinir ve bellekteki kaynaklar serbest bırakılır.

EBO.cpp

OpenGL'de kullanılan Element Buffer Object (EBO) işlemlerini yöneten bir sınıftır. EBO, vertex verilerinin çizim sırasında tekrar kullanılmasını sağlar. Bu sayede tekrarlayan vertex'ler için bellekte gereksiz veri tutulmaz ve performans artar.

Constructor (EBO::EBO): Dışarıdan bir `std::vector<GLuint>` alınır ve bu index verileri GPU'ya aktarılır.

- `glGenBuffers` ile bir EBO ID'si oluşturulur.
- `glBindBuffer` ile bu ID'ye sahip EBO `GL_ELEMENT_ARRAY_BUFFER` hedefiyle bağlanır.
- `glBufferData` fonksiyonu ile index verileri GPU'ya kopyalanır.

Bind(): EBO'yu GL_ELEMENT_ARRAY_BUFFER olarak bağlar. Bu işlem, EBO'nun aktif hale gelmesini sağlar.

Unbind(): glBindBuffer fonksiyonu ile 0 ID'si atanarak aktif EBO çözülür. Böylece başka işlemlerle çakışmaması sağlanır.

Delete(): glDeleteBuffers ile oluşturulan EBO silinir ve bellekteki kaynaklar serbest bırakılır.

VAO.cpp

OpenGL'de kullanılan Vertex Array Object (VAO) işlemlerini yöneten bir sınıftır. VAO, VBO ve EBO gibi buffer'ların yapılandırılmalarını saklar ve bu sayede vertex attribute'larının nasıl okunacağını tanımlar. VAO kullanımı, çizim işlemlerinde kodun okunabilirliğini ve verimliliğini artırır.

Constructor (VAO::VAO): VAO oluşturmak için kullanılır.

- glGenVertexArrays fonksiyonu ile bir VAO ID'si oluşturulur.

LinkAttrib():

- VBO'da tanımlı olan bir attribute'u (örneğin pozisyon, renk, doku koordinatı gibi) VAO ile ilişkilendirmek için kullanılır.
- VBO.Bind() ile ilgili VBO bağlanır.
- glVertexAttribPointer fonksiyonu ile attribute bilgisi tanımlanır (kaç bileşen olduğu, tipi, stride, offset vs.).
- glEnableVertexAttribArray ile ilgili attribute etkinleştirilir.
- VBO.Unbind() ile VBO çözülür.

Bind(): glBindVertexArray ile VAO aktif hale getirilir.

Unbind(): glBindVertexArray(0) ile aktif VAO çözülür.

Delete(): glDeleteVertexArrays ile VAO silinir ve kaynaklar serbest bırakılır.

texture.cpp

OpenGL'de bir görsel dosyasını texture olarak yüklemek, bağlamak ve yönetmek için kullanılan Texture sınıfıdır. Projede STB_Image kütüphanesi kullanılarak görseller byte formatında okunur ve OpenGL'in texture yapısına entegre edilir.

Constructor (Texture::Texture): Dışarıdan bir görsel dosya yolu (image), texture türü (texType), bağlı olacağı texture birimi (slot), format ve piksel tipi alınır. Görsel dosya STB_Image ile yüklenir ve OpenGL'e texture olarak aktarılır.

- `stbi_set_flip_vertically_on_load(flip)` ile görselin dikey ekseninde çevrilip çevrilmeyeceği ayarlanır.
- `stbi_load` fonksiyonu ile görsel byte dizisine dönüştürülür.
- `glGenTextures` ile bir texture ID'si oluşturulur.
- `glBindTexture` ile texture bağlanır.
- `glTexParameteri` ile texture'ün filtreleme ve sarmalama ayarları yapılır (örneğin tekrar etme veya kenarda sabitleme).
- `glTexImage2D` ile byte dizisi OpenGL'e yüklenir.
- `glGenerateMipmap` ile mipmap'ler otomatik olarak oluşturulur.
- Görsel verisi RAM'den silinir çünkü artık GPU'dadır.
- Texture unbind edilir.

texUnit(): Shader içindeki bir uniform değişkene bu texture'ın bağlı olduğu birimi (unit) atamak için kullanılır.

- `glGetUniformLocation` ile uniform'un lokasyonu alınır.
- Shader aktive edilir.
- `glUniform1i` ile uniform'a birim atanır.

Bind() Function: Texture'ü aktif hale getirir ve GL_TEXTURE_2D olarak bağlar.

Unbind() Function: Texture'ün bağını keser

Delete() Function: Texture GPU'dan silinir ve bellekteki kaynaklar serbest bırakılır.

camera.cpp

Sahnedeki kameranın konumunu, yönünü ve kullanıcı etkileşimlerini yöneten Camera sınıfıdır. Bu sınıf sayesinde 3D sahnede serbest bir şekilde dolaşmak, bakış yönünü değiştirmek ve etkileşimli bir kamera kontrolü sağlamak mümkün olur.

Constructor (Camera::Camera): Kameranın başlangıç boyutları (width, height) ve konumu (position) atanır. Bu değerler kameranın yerleşimi ve bakış yönü hesaplamaları için temel oluşturur.

updateMatrix(): Kamera matrisi oluşturulur. Bu matris, görünüm (view) ve projeksiyon (projection) matrislerinin çarpımıdır.

- `glm::lookAt()` ile kameranın baktığı yön ve pozisyon belirlenir.
- `glm::perspective()` ile sahneye perspektif eklenir (FOV, oran, near/far plane).
- `cameraMatrix`, bu iki matrisin çarpımı olarak güncellenir.

Matrix(): `cameraMatrix` shader programına gönderilir.

- `glGetUniformLocation` ile uniform lokasyonu alınır.
- `glUniformMatrix4fv` ile `cameraMatrix`, shader'da ilgili uniform'a atanır.

getViewMatrix() / getProjectionMatrix(): Kamera görünüm ve projeksiyon matrislerini tekil olarak döndürür. Dışarıdan da kullanılabilecek şekilde tanımlanmıştır.

Inputs(): Kullanıcıdan gelen klavye ve mouse girişlerini işler.

- W, A, S, D tuşları ile ileri, geri, sola, sağa hareket.
- SPACE ve LEFT CONTROL ile yukarı-aşağı hareket.
- LEFT SHIFT basılı tutulduğunda hareket hızı artırılır.
- Sol mouse tuşuna basıldığında kamera dönüşü aktif hale gelir.
- İmleç ekranın ortasında tutulur ve hareket farkı ile kamera yönü döndürülür (rotX, rotY).
- `glm::rotate()` ile yön vektörü (Orientation) güncellenir.

model.cpp

Bir 3D modelin tepe noktaları (vertex), indeksleri ve varsa dokularıyla (texture) birlikte GPU'ya gönderilip çizilmesini sağlar.

Constructor (Mesh::Mesh): vertices, indices, textures gibi model verilerini alır. VAO, VBO ve EBO oluşturulur. Vertex verileri VAO'ya konum, normal, renk ve doku bilgisi olarak bağlanır. Doku kullanılıp kullanılmayacağı isTexture ile belirlenir.

Çizim Fonksiyonu (Draw): Shader aktif hale getirilir, VAO bağlanır. Eğer dokular varsa shader'a atanır ve GPU'ya yüklenir. Kamera pozisyonu ve matrisi shader'a gönderilir. `glDrawElements` ile üçgenler ekrana çizdirilir.

shaderClass.cpp

Vertex ve fragment shader dosyalarını okuyup derleyerek GPU üzerinde çalışan bir shader programı oluşturur, bu programı aktif hale getirir veya siler.

Dosya Okuma (get_file_contents): Belirtilen dosya yolundan tüm içerik okunarak std::string olarak döndürülür. Shader kaynak kodlarını okumak için kullanılır.

Constructor(Shader::Shader): Vertex ve fragment shader dosyaları okunur, içerikleri const char* olarak hazırlanır. Ardından bu kaynaklar kullanılarak OpenGL üzerinde vertex ve fragment shader nesneleri oluşturulup derlenir. Shader'lar bir program altında birleştirilir ve bu program kullanılmaya hazır hale getirilir.

Aktifleştirme ve Silme (Activate, Delete): Activate() fonksiyonu shader programını aktif hale getirir. Delete() fonksiyonu ise GPU'daki programı temizler.

Hata Kontrolü (compileErrors): Hem derleme (shader bazında) hem de bağlama (program bazında) sırasında oluşabilecek hataları yakalar ve anlamlı hata mesajları olarak ekrana yazdırır.

Shaders

Vertex Shader(default.vert)

Bu shader ile modelin vertex'lerini dünya koordinatlarına dönüştürülür ve diğer bilgileri fragment shader'a gönderilir.

layout location:

- `aPos`, `aNormal`, `aColor`, `aTex` – sırayla pozisyon, yüzey normali, vertex rengi ve doku koordinatlarıdır.

Uniformlar:

- `model` – model matrisidir, objenin kendi koordinat sisteminden dünya sistemine dönüşümünü sağlar.
- `camMatrix` – kamera bakış açısı ve perspektif bilgilerini içerir.

Çıktı (out):

- `crntPos` – dünya uzayındaki vertex pozisyonudur.
- `Normal`, `color`, `texCoord` – hesaplamalarda kullanılmak üzere fragment shader'a gönderilir.

Fragment Shader(default.frag)

Daha gerçekçi ışıklandırma efekti uygulamak için kullanılır. Üç farklı ışık türü hesaplaması yapılır: pointLight, direcLight ve spotLight.

Girdi (in):

- `crntPos`, `Normal`, `color`, `texCoord` – vertex shader'dan gelen pozisyon, yüzey normali, renk ve doku koordinatlarıdır.

Uniformlar:

- `diffuse0`, `specular0` – iki ayrı doku.
- `lightColor`, `lightPos`, `camPos` – ışık rengi, konumu ve kamera pozisyonu.

Işık türleri:

- `pointLight()` – noktasal ışık. Mesafeye göre ışık zayıflar.
- `direcLight()` – yönlü ışık. Sonsuz uzaklıktan gelen paralel ışınlar gibi davranır.
- `spotLight()` – belirli açıda koni şeklinde yayılan ışık. İç-dış koni açısına göre etki gücü azalır.

Işık bileşenleri:

- Ambient Light: Ortamda her yere eşit yayılan sabit ışık.
- Diffuse Light: Işık, yüzeye dik geldikçe artar.
- Specular Light: Parlak nokta efekti. Kamera bakış açısına göre hesaplanır.

Modeller ve OpenGL Fonksiyonları(Main.cpp)

GLFW ve OpenGL Başlatma: İlk olarak, glfwInit() fonksiyonu ile GLFW (grafik penceresi ve kullanıcı etkileşimi için bir kütüphane) başlatılıyor. Sonrasında OpenGL versiyonunu ayarlıyoruz (3.3 sürümünü seçiyoruz). Eğer pencereyi oluşturamazsak, hata mesajı verip programdan çıkıyoruz.

Texture Yükleme: Burada, çeşitli objelere (yatak, masa, halı, bilgisayar gibi) uygulanacak olan doku (texture) dosyalarını yüklüyoruz. Her objeye ait bir doku (örneğin, floor.jpg, carpet.jpg) dosyasını yükleyip, her objeye bu dokuyu atıyoruz. Bunlar daha sonra objelerin yüzeylerine yerleştirilecek.

```
// Texture datas
Texture floorTexture("images/floor.jpg", "diffuse", 0, GL_RGB, GL_UNSIGNED_BYTE, true);
std::vector<Texture> floorTextures = { floorTexture };

Texture carpetTexture("images/carpet.jpg", "specular", 0, GL_RGB, GL_UNSIGNED_BYTE, true);
std::vector<Texture> carpetTextures = { carpetTexture };
```

Objelerin Mesh'lerini Oluşturma: Burada, sahnedeki her objenin geometrik şekli (mesh) oluşturuluyor. Örneğin, bir masa, bir yatak, bir bilgisayar gibi objeler için vertex (nokta) verileri ve bu verileri birleştiren indeksler oluşturuluyor. Bunlar daha sonra bir Mesh objesi haline geliyor.

```
// Floor Mesh'i
std::vector<Vertex> verts(vertices, vertices + sizeof(vertices) / sizeof(Vertex));
std::vector<GLuint> ind(indices, indices + sizeof(indices) / sizeof(GLuint));
Mesh floor(verts, ind, floorTextures, true);

// Arka duvar Mesh'i
std::vector<Vertex> backVerts(backWallVertices, backWallVertices + sizeof(backWallVertices) / sizeof(Vertex));
std::vector<GLuint> backInd(backWallIndices, backWallIndices + sizeof(backWallIndices) / sizeof(GLuint));
Mesh backWall(backVerts, backInd, wallTextures, true);

// Sol duvar
std::vector<Vertex> leftVerts(leftWallVertices, leftWallVertices + sizeof(leftWallVertices) / sizeof(Vertex));
std::vector<GLuint> leftInd(leftWallIndices, leftWallIndices + sizeof(leftWallIndices) / sizeof(GLuint));
Mesh leftWall(leftVerts, leftInd, wallTextures, true);
```

Shader Programları: lightShader ve shaderProgram olmak üzere 2 farklı shader kullanılmıştır. Shader'ların uniform değişkenlerine gerekli bilgilerin iletilmesi ve shader'ların aktive edilmesi yapılmıştır.

```
glm::vec4 lightColor = glm::vec4(1.0f, 1.0f, 1.0f, 1.0f);
glm::vec3 lightPos = glm::vec3(0.0f, 2.0f, 0.0f);
glm::mat4 lightModel = glm::mat4(1.0f);
lightModel = glm::translate(lightModel, lightPos);

glm::vec3 objectPos = glm::vec3(0.0f, 0.0f, 0.0f);
glm::mat4 objectModel = glm::mat4(1.0f);
objectModel = glm::translate(objectModel, objectPos);

lightShader.Activate();
glUniformMatrix4fv(glGetUniformLocation(lightShader.ID, "model"), 1, GL_FALSE, glm::value_ptr(lightModel));
glUniform4f(glGetUniformLocation(lightShader.ID, "lightColor"), lightColor.x, lightColor.y, lightColor.z, lightColor.w);
shaderProgram.Activate();
glUniformMatrix4fv(glGetUniformLocation(shaderProgram.ID, "model"), 1, GL_FALSE, glm::value_ptr(objectModel));
glUniform4f(glGetUniformLocation(shaderProgram.ID, "lightColor"), lightColor.x, lightColor.y, lightColor.z, lightColor.w);
glUniform3f(glGetUniformLocation(shaderProgram.ID, "lightPos"), lightPos.x, lightPos.y, lightPos.z);
```

Main Loop: Bu kısım sürekli çalışacak döngüdür. Her döngüde ekran temizleniyor. Kameradan gelen kullanıcı girişleri (klavye, fare hareketleri) alınıyor ve kameranın konumu güncelleniyor. Sonrasında, tüm objeler sırasıyla çiziliyor. Yani, önce zemin, sonra halı, duvarlar, masa, bilgisayar gibi objeler sırasıyla sahnede çiziliyor.

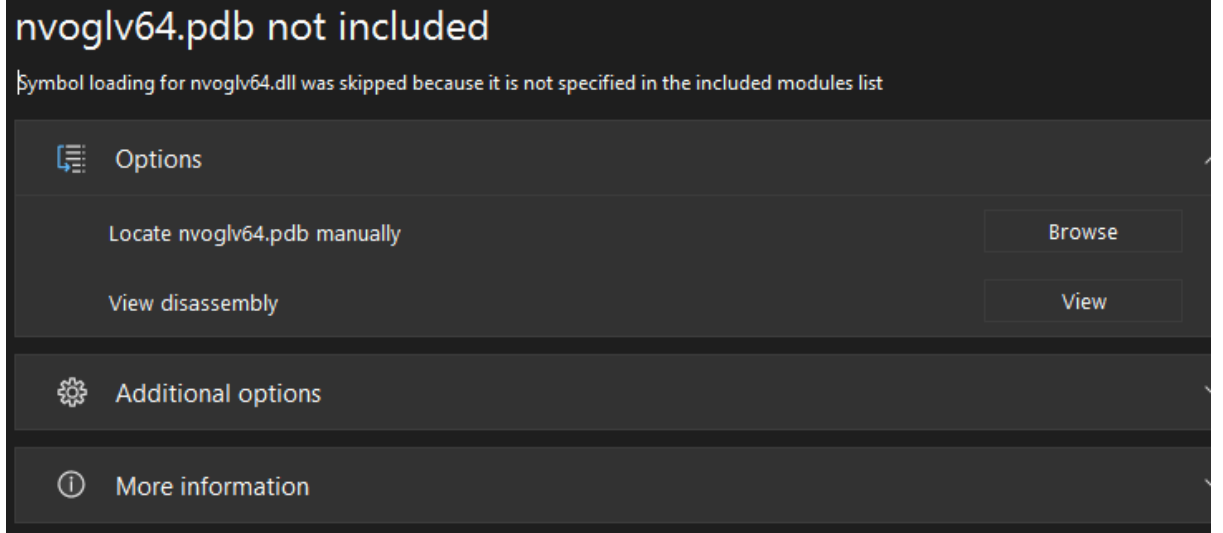
```
// Draws different meshes
floor.Draw(shaderProgram, camera);
carpet.Draw(shaderProgram, camera);
light.Draw(lightShader, camera);
backWall.Draw(shaderProgram, camera);
leftWall.Draw(shaderProgram, camera);
rightWall.Draw(shaderProgram, camera);
frame.Draw(shaderProgram, camera);
frameImage.Draw(shaderProgram, camera);
bed.Draw(shaderProgram, camera);
bedHeader.Draw(shaderProgram, camera);
bedPart.Draw(shaderProgram, camera);
table.Draw(shaderProgram, camera);
leftlegtable.Draw(shaderProgram, camera);
rightlegtable.Draw(shaderProgram, camera);
```

Bitiş: Uygulama kapanmadan önce her şey temizleniyor. OpenGL kaynakları siliniyor ve pencere kapatılıyor.

```
shaderProgram.Delete();
lightShader.Delete();
glfwDestroyWindow(window);
glfwTerminate();
return 0;
```

Karşılaşılan problemler ve çözümler

1. 'nvoglv64.pdb' hatası: Yüklemediğim görüntülerin 3 ya da 4 kanallı olmasından kaynaklanıyormuş. 3 kanallı bir görüntü için parametre olarak 'GL_RGBA' verdiğimde ya da 4 kanallı bir görüntü için 'GL_RGB' verdiğimde bu hata meydana geliyor.



2. Bazı görüntülerin ters yüklendiğini fark ettim. Araştırdığımda stb kütüphanesine bağlı 'stbi_set_flip_vertically_on_load' fonksiyonu olduğunu gördüm. Bu fonksiyon ile görüntü ilk yüklendiğinde ters çevrilip görüntüleniyor. Böylece görselin ortamda ters gözükmesi engelleniyor.

Kaynaklar

- Victor Gordan tarafından yayınlanan tutorial izledim. Konuyu güzel ve net anlatmış.
<https://www.youtube.com/watch?v=XpBGwZNyUh0&list=PLPaoO-vpZn umdcb4tZc4x5Q-v7CkrQ6M->
- <https://learnopengl.com/Getting-started/Textures>